

KOGI

Independent Worker Operating System

Unified Platform Design Document

"One Portfolio. Infinite Possibilities."

This document is the definitive unified design specification for the entire Kogi Platform — consolidating system architecture, module designs, data models, engine specifications, AI workflows, game mechanics, community systems, banking and capital infrastructure, tools, and product roadmap into a single reference.

SECTION 1 · VISION, MISSION & DESIGN PRINCIPLES

1. Vision & Mission

Kogi is the Independent Worker Operating System — a complete, intelligent, unified platform that elevates the autonomy, creativity, financial stability, and community connection of every independent worker on the planet.

Brand Idea
Empower the Independent.

1.1 Mission Statement

Provide every independent worker with a complete, intelligent, unified portfolio platform that elevates their autonomy, creativity, financial stability, and connection to communities and opportunities. The entire system is anchored around one independent worker's single portfolio — built to scale outward into networks, communities, and marketplaces — while always preserving autonomous ownership, granular control, and centralized governance.

1.2 Brand Values

Value	Expression
Independence	Every feature preserves the worker's right to own, control, and move their data, income, and reputation freely.
Clarity	Complex systems surface as simple, actionable interfaces. Intelligence is invisible until it's needed.
Intelligence	AI continuously works in the background so the worker can focus on their craft and goals.
Flow	Work, finance, community, and growth are connected in a single uninterrupted workflow.
Integrity	All transactions, governance, and attributions are transparent, auditable, and tamper-evident.
Empowerment	Every tool is designed to increase the worker's capacity, leverage, and options.
Community	Individual portfolios connect into cooperative networks that amplify collective power.

1.3 Brand Taglines

- "One Portfolio. Infinite Possibilities."
- "Your independent work, unified."

- "The platform for independent ambition."

2. Design Principles

2.1 Centered on the Single Independent Worker

The platform centers the single portfolio for the single independent worker. Connecting, sharing, extending, and collaborating with other workers and communities is supported, but the individual portfolio is always the atomic unit of the system. All collective, cooperative, and organizational features are additive layers on top of the sovereign individual portfolio.

2.2 Unified, Coherent Architecture

Every component is interconnected and consistent in UX, operational model, and data primitives. All components are backed by shared platform services and extensible through a unified developer API. There are no data islands — information flows freely across modules through the shared Portfolio Component abstraction.

2.3 Universal Modularity

Every module, primitive, and artifact supports: configuration, versioning, and extension/plug-in model; theming, templates, automation, and APIs; full lifecycle management from create through archive and restore; metadata tracking on every entity; and Minimal Valuable Elements — the minimal viable unit of every component or primitive in the platform.

2.4 AI-Driven Operational Intelligence

A platform-wide AI Intelligence & Cognition Engine (kogi-engine) provides optimization, portfolio insights, learning and adaptation, automated workflows, recommendations and matching, risk and resilience management, and intelligent agents. The Oba assistant runs on the Sambara agent system and ImaniOS, and acts as each user's personal chief-of-staff across every module.

2.5 Comprehensive Automation

Workflow, orchestration, and process automation runs across all modules — including event-driven automation, scheduled automation, trigger-based actions, and cross-component orchestrations. Consequential actions always require user approval; repetitive low-stakes actions can be fully automated by Oba.

2.6 Integrated Feed & Activity System

A platform-wide feed service powers activity streams, notifications, alerts, real-time updates, and personalized AI-curated dashboards. Every module publishes events to the shared feed substrate, which the PersonalizationEngine ranks and delivers per user.

2.7 Universal Platform Properties

Every platform component, every primitive, and every artifact is: modular, extensible, configurable, manageable, and administratable; auditable and compliant; scalable, secure, and lifecycle-managed; archivable, restorable, recoverable, and resilient; optimizable, monitorable, and maintainable; and equipped with trackable metadata backed by an append-only EventLog.

2.8 Approval-First by Default

Every consequential action — sending money, submitting governance proposals, contacting external parties, executing batch payouts — requires explicit user confirmation. Agents suggest and prepare; users decide. All agent actions are logged with timestamps, tool names, inputs, and outcomes, viewable from the History panel at any time.

SECTION 2 · PLATFORM ARCHITECTURE

3. Platform Architecture Overview

The Kogi platform is organized as a layered operating system. KOGI-OS hosts all user-facing applications. KOGI-ENGINE provides the intelligence, data processing, and automation substrate. KOGI-BASE manages physical infrastructure, security, and persistence. KOGI-APPSTORE enables extension distribution. KOGI-MANAGER provides governance and RBAC administration.

Layer	System	Responsibility
Application Layer	KOGI-OS	Portal · Apps · SDK · Host · All user-facing product applications
Intelligence Layer	KOGI-ENGINE	AI · Data Platform · Automation · Controls & Optimization · Scala engine services
Infrastructure Layer	KOGI-BASE	Infrastructure · Servers · Devices · Security · Backup & Recovery
Distribution Layer	KOGI-APPSTORE	Apps · Plugins · Templates · Add-ons · Community extensions
Administration Layer	KOGI-MANAGER	Admin · Config · Governance · RBAC · Policy management

3.1 Technology Stack

Component	Language / Technology	Role
kogi-kernel	Zig	Low-level memory, process, file, resource management and scheduling
kogi-host	Rust	Platform host — central executive control, module orchestration
kogi-server	Rust	Backend server connecting kernel, modules, host, and clients
kogi-services	Go	Microservices: networking, messaging, pub/sub, gateway management
kogi-engine	Scala 3 (JVM)	Data engine: analytics, AI, matching, risk, graph, search, allocation
kogi-modules	Rust (core) + Go (services)	Office, Bank, Exchange, Marketplace, Community, Studio modules
kogi-database	PostgreSQL + Redis + Kafka	Persistence, caching, and event streaming
kogi-web-client	Angular + TypeScript	Web frontend

Component	Language / Technology	Role
kogi-desktop-client	Java	Desktop client application
kogi-mobile	Kotlin + iOS Swift	Android and iOS mobile clients
Build System	Bazel	Unified build environment across all components

3.2 Platform Data Flow

User Actions → API Gateway → Microservices → DB / Event Bus → Feed Hub → AI Engine → Dashboards / Recommendations → Workflow Automation → User

3.3 Distributed Node Architecture

The Kogi platform scales through a distributed node system. Multiple kogi-hosts can be instantiated and each acts as a node in a kogi-network. Services scale through multiple instances acting as additional nodes. Hosts act as centralizing nodes where services attach and communicate to/with specific hosts, while the network module orchestrates all networking for that host and its attached services.

Connection between hosts is decentralized: multiple hosts coordinate to load-balance the platform workload. A leader host is elected when multiple hosts run as a grouped instance, using algorithms such as Raft or Bully for consensus. All nodes send heartbeats when running and connected to the kogi-network, with health-check and node-status messages. Federates register with one another to indicate joining the platform federation.

3.4 ShangoOS & Federation Context

When Kogi (personal work OS), qala (product/solution development OS), ume (business and legal entity OS), oru (simulation and strategy platform), sambara (agent coordinator), imewe (automated fabrication), nandi (automated mobility), and osyse (infrastructure management) work together, they form ShangoOS. A jiwe seed boot image can configure all these systems as a single unified system to achieve specified outcomes. The Pamoja Federation is one such configuration — a cyberphysical system running as a digital twin on ShangoOS.

4. KOGI-OS — Central Platform Operating System

KOGI-OS centralizes, contains, and maintains all platform component applications and the application ecosystem. It provides a unified interface into the entire system through the KOGI-Portal, KOGI-Host Kernel, and KOGI-SDK.

Sub-System	Description
KOGI-Portal	Unified UI/dashboard — the single entry point for all platform apps, feeds, dashboards, and notifications
KOGI-Host Kernel	Platform kernel/core managing security, identity, RBAC, core primitives, alerts/triggers, monitoring

Sub-System	Description
KOGI-SDK	Unified developer API, extension framework, and plug-and-play runtime for adding future applications

4.1 Component Applications Hosted in KOGI-OS

kogi-office · kogi-bank · kogi-exchange · kogi-marketplace · kogi-community · kogi-studio · kogi-dev · kogi-profile · kogi-organizations

4.2 KOGI-HOST Architecture

kogi-host is the central executive control system. Services attach to and communicate with specific hosts. The host coordinates modules, services, the server, and the engine. It exposes an interactive shell/CLI for accessing and checking all components. It broadcasts heartbeats, manages node state, and participates in leader election for multi-host deployments.

SECTION 3 · PORTFOLIO SYSTEM

5. Portfolio System — kogi-portfolio

Design Principle

Everything in the Kogi ecosystem is a Portfolio Item. The Portfolio System is therefore the universal abstraction layer that every platform application builds upon.

The Kogi Portfolio System (portfolio_system.rs, v2.1.0) is the central domain engine — the single source of truth for every entity belonging to an independent worker's portfolio. It defines the full data model, lifecycle rules, distribution strategy, and computational models that underpin kogi-portfolio and every other Kogi application.

5.1 Key Design Goals

- Universal abstraction — every platform entity is representable as a Component (Item or Container)
- Event-sourced, auditable — every mutation appended to an immutable EventLog (capped at 10,000 events; older events flushed to Data Lake)
- Distributed-first — VectorClock + CRDT log (LWW + OR-Set) for multi-node / federated operation
- Permission-gated — all mutations validated against a tiered PermissionTier hierarchy
- Policy-pluggable — governance PolicyEngine trait allows platform-level and custom rules
- Extensible — PortfolioPlugin trait + ToolBox integration (TMS) for runtime extension
- Computationally rich — 11 built-in analytical models covering health, risk, resource, and maturity

5.2 Structural Hierarchy

The top-level architecture: PortfolioSystem → Component (Item | Container) → ComponentMetadata + ComponentData → component:item and component:container. Supporting structures include: GraphEdge, EventLog, CrdtLog, VectorClock, Structural Primitives, Governance (PolicyEngine, ApprovalWorkflow, ResourceAllocation), Plugins, Federation, and Computational Models.

5.3 Component Taxonomy — Items

Item Type	Payload Struct	Key Fields
Portfolio	PortfolioItemPayload	mission, focus_areas, kpis: Vec<Metric>
Program	ProgramPayload	objective, projects: Vec<ComponentId>, budget
Project	ProjectPayload	project_type, methodology, sprints, backlog, releases
Resource	ResourcePayload	resource_type, skills, availability

Item Type	Payload Struct	Key Fields
Artifact	ArtifactPayload	artifact_type, content, version, format, file_size
Asset	AssetPayload	asset_type, valuation, ownership, depreciation
SubPortfolio	SubPortfolioPayload	theme, scope, rollup_metrics

5.4 Component Taxonomy — Containers

Container Type	Description
Binder	Structured knowledge container — SOP, process, reference library
Book	Narrative or sequential document container
Record	Structured data record with schema
Folder	General-purpose file container
Registry	Enumeration or catalog container
Archive	Deep storage for completed or deprecated items

5.5 Graph Edges

Edge Type	Meaning
Hierarchy	Parent-child containment relationship
Dependency	Work item depends on another item
Link	Lateral reference between related items
Contains	Container holds a component
Federation	Cross-system / cross-platform relationship

5.6 Computational Models

Model	Purpose
PortfolioHealth	Overall portfolio health scoring across all dimensions
ProjectMetrics	Velocity, throughput, cycle time, lead time, delivery confidence
ProgramAlignment	Cross-project alignment to strategic objective
SubPortfolioRollup	Aggregated metrics across a sub-portfolio group
ResourceUtilisation	Allocation vs. capacity across human and physical resources
AssetValue	Current valuation, depreciation, and ROI tracking
ArtifactMaturity	Content completeness, review status, and publication readiness
BinderCoverage	Documentation coverage and gap detection
BookConsistency	Narrative consistency and version coherence

Model	Purpose
FolderOrganisation	Structural health and findability of folder hierarchies
RecordIntegrity	Schema compliance and data quality for structured records

5.7 Core API Surface

Method Group	Key Methods
Component CRUD	create_item() · create_container() · read() · read_mut() · update_info() · delete()
Graph	attach_child() · detach_child() · add_dependency() · add_link()
Actions	act(user, component_id, ActionKind)
Analytics	compute_portfolio_health() · compute_project_metrics() · compute_program_alignment() · compute_sub_portfolio_rollup()
Version Control	bump_version(actor, id, VersionPart, message)
Resource	allocate_resource() · allocate_budget() · record_consumption() · record_spend() · overrun_allocations()
Governance	register_policy_engine() · attach_policy() · detach_policy() · request_approval() · resolve_approval()
Snapshots	snapshot() · save_snapshot() · save_checkpoint() · restore_snapshot()
CRDT	apply_crdt_log() · crdt_log()
TMS	attach_toolbox() · detach_toolbox() · has_toolbox() · toolbox_ids()
Plugins	register_plugin() · unregister_plugin()
Search/Query	search() · query()
Structural	create_group() · create_collection() · create_list() · create_schedule() · create_directory()

SECTION 4 · RESOURCE MANAGEMENT SYSTEM (KOGI-RMS)

6. Resource Management System

KOGI-RMS is the unified layer for discovering, organizing, allocating, governing, and tracking every resource on the Kogi platform. A resource is any entity — structured or unstructured, human or non-human, tangible or intangible — that has value, can be acted upon, and can be associated with a Portfolio component. KOGI-RMS is not a separate application; it is the resource substrate that underlies every other Kogi module.

6.1 Resource Domain Coverage

Domain	Resource Categories Covered
Portfolio & Work	Portfolios, programs, projects, assets, artifacts, binders, journals, books, dossiers, folders, documents, files, directories
Planning & Time	Timelines, schedules, roadmaps, calendars, Gantt charts
Work Execution	Boards, epics, stories, features, tasks, work packages, WBS items, initiatives, strategies, themes
Commerce & Contracts	Gigs, bookings, consultations, jobs, contracts, offers, deals, requests, proposals, bids, investments
Capital & Physical	Capital, financial assets, artifacts, labor, land, estates, real estate
Finance & Instruments	Equity, liquidity, debt, taxes, cash, credit, debit, donations, grants
Contributions	Labor, skills, financial, support, advertising, marketing, promotions, endorsements, donations, investments
Users & Roles	Members, contributors, donors, investors, subscribers, followers, watchers, owners, editors
Organizations	Autonomous orgs, open source communities, cooperatives, collectives, federations, councils, assemblies

6.2 Resource Base Schema

Field	Type	Description
resource_id	UUID	Platform-wide unique identifier; immutable
resource_type	ResourceType	Top-level classification — see taxonomy
resource_subtype	String	Domain-specific subtype (e.g., Portfolio, Gig, HSA, Land)
name	String	Display name
slug	String	URL-safe unique handle
description	String	Rich-text description; markdown supported

Field	Type	Description
status	ResourceStatus	Draft Active Paused Archived Deleted
visibility	Visibility	Public Private Protected Space Organization Invite
owners	Vec<EntityId>	One or more users or orgs with Owner privilege
editors	Vec<EntityId>	Users or orgs with Edit privilege
tags	Vec<String>	Hashtag-style discovery tags; full-text indexed
labels	Vec<String>	Structured classification labels; filterable
policy_ids	Vec<PolicyId>	Governance policies; inherited from parent unless overridden
parent_ids	Vec<ComponentId> >	Parent resources in hierarchy
children_ids	Vec<ComponentId> >	Child resources in hierarchy
vector_clock	VectorClock	CRDT conflict resolution; auto-managed by platform
event_log	Vec<EventRecord>	Append-only audit history of all mutations
analytics_id	AnalyticsRef	Pointer to AnalyticsEngine time-series for this resource

6.3 Universal Resource Actions

Action Group	Actions
Discovery	search · filter · index · tag · label · mention · hashtag · topic · recommend · discover · explore
Social	like · comment · share · follow · subscribe · watch · save · bookmark · poll · survey · invite
Content	view · preview · download · fork · clone · embed · export · print · archive
Governance	own · edit · CRUD · post · report · flag · approve · reject · propose · vote
Commerce	donate · invest · fund · bid · offer · deal · trade · allocate · contribute
Lifecycle	create · draft · publish · pause · unpublish · delete · restore · version · snapshot
Collaboration	assign · delegate · request · respond · notify · alert · broadcast · message

SECTION 5 · WORK MANAGEMENT SYSTEM (KOGI-WMS)

7. Work Management System

The Kogi Work Management System (KOGI-WMS) is the planning, execution, and coordination layer of the Kogi Independent Worker Operating System. It provides independent workers, collectives, cooperatives, and autonomous organizations with a unified workspace for managing every unit of work — from high-level portfolio themes down to individual tasks. KOGI-WMS is designed around the Work Breakdown Structure (WBS) model, ensuring all work is traceable from strategic vision to tactical execution.

7.1 WBS Hierarchy

Level	Element	Description
L1	Theme	Strategic direction or domain-level focus area
L2	Initiative	Cross-epic strategic effort aligned to a theme
L3	Epic	Large body of work, decomposed into stories; fits within a quarter or PI
L4	Story	Primary deliverable unit: estimable, testable, completable within a sprint
L5	Task	Atomic unit of execution within a story; completed within hours or a day

7.2 Workspace Modules

Module	Purpose
Workspace	Central hub: dashboard, boards, timelines, analytics, resource management, content
Work Breakdown Structure	Hierarchical decomposition: Theme → Initiative → Epic → Story → Task
Work Studio	Requirements management and work design tooling
Governance	Policies, approvals, access control, audit trails
Analytics & Optimization	Forecasting, KPIs, OKRs, telemetry, performance dashboards
Resource Management	Budgeting, allocation, delegation, reporting
Content Management	Files, documents, contracts, SOPs, policies, frameworks

7.3 Work Dashboard Widgets

Widget	Description	Data Source
Active Items Summary	Count of open themes, initiatives, epics, stories, tasks with status distribution	WBS Engine

Widget	Description	Data Source
Priority Queue	Top 5–10 highest-priority in-progress items sorted by urgency and dependency	OptimizationEngine
Velocity Tracker	Story points / items completed per sprint with trend line	AnalyticsEngine
Blocker Alerts	Real-time flags on blocked stories and unresolved dependencies	WBS Engine + Oba
Upcoming Milestones	Next 30-day milestone and goal timeline	BookingEngine + Timeline
Team Capacity	Available vs. allocated capacity by member and role	AllocationEngine
OKR Progress	Key results progress meters per active Objective	GoalEngine

7.4 Backlog Types

Backlog Type	Scope & Description
Program Backlog	All epics and stories across all projects within a program; managed by Program Lead
Project Backlog	All stories and tasks scoped to a single project; managed by Project Owner
Sprint / Timebox Backlog	Stories committed to the current sprint or PI increment
Team Backlog	Personal or team-specific task queue, filtered from the project backlog
Triage Queue	Ungroomed submissions (bugs, requests, ideas) awaiting refinement and prioritization
Archive	Closed, cancelled, or deferred items retained for historical analysis

7.5 Work Item Types

Type	Category	Description
Feature	Delivery	New user-facing capability or product increment
Bug	Quality	Defect in existing behavior causing incorrect output
Capability	Delivery	Technical or platform enabler delivering underlying functionality
Enhancement	Delivery	Improvement to existing functionality rather than net-new feature
Innovation	Strategy	Exploratory work: prototype, spike, experiment, or innovation sprint item
Blocker	Operations	Item whose resolution is blocking other items; requires escalated attention
Risk	Governance	Identified risk with likelihood, impact, and mitigation plan

Type	Category	Description
Strategy	Strategy	Strategic direction or approach document; parent of initiatives
Milestone	Planning	A significant event or decision point in a program or project
Objective	Planning	OKR Objective — aspirational direction for a team or program
Release	Delivery	A versioned set of work items bundled for deployment to users

SECTION 6 · PLATFORM MODULE MAP — DESIGN.MD

8. Platform Navigation & Feature Map

The Kogi platform is organized into four top-level navigation modules: Home, Office, Community, and Bank. Each module contains multiple feature areas, subsystems, and engines. This section provides the definitive feature map for the entire platform.

8.1 Home Module

8.1.1 Portable Benefits

Portable benefits are worker-centered benefits that remain with the individual rather than being tied to a single employer, designed specifically for independent contractors, freelancers, and gig workers.

Category	Benefit Types
Health & Wellness	Health insurance, dental, vision, Health Savings Accounts (HSA)
Retirement & Savings	SEP-IRA, 401k portability, Pooled Employer Plans (PEP), emergency savings accounts
Paid Time Off & Income Security	Paid sick days, vacation time, income replacement for missed work
Insurance & Protection	Occupational accident insurance, disability, workers' compensation
Professional Development	Portable education accounts, training fund, certification credits
Platform Contributions	Gig platform % contributions (e.g., 4% of pre-tip earnings) to portable savings accounts, managed by benefit providers (e.g., Stride LLC)

8.1.2 Initiatives

Initiative	Description
Portable Benefits	Coordinate worker groups for pooled benefits purchasing power and PEP enrollment
Grants & Microfinancing	Grant discovery, application, community lending pools, microloan origination
Group Economics	Cooperative treasury, revenue sharing, group investment, mutual aid funds
Equity Crowdfunding	Reg CF/A+ campaigns, revenue share, SAFE notes, cap table management
Crowdresourcing	Community-sourced labor, capital, asset, knowledge contributions
Shared Portfolios	Multi-user, multi-org portfolio management with attribution and governance

Initiative	Description
Portfolio Resource Sharing	Share artifacts, templates, tools, playbooks, and data within community contexts

8.2 Office Module

8.2.1 Booking & Scheduling

Feature	Description
Centralized Calendars	Unified calendar aggregating bookings, events, milestones across office and community
Multi-Worker Dashboards	Overview of all member schedules, availability, and booking conflicts
Conflict Detection	Automated scheduling conflict detection; Oba alerts before confirmation
Event Booking & Reservations	Full lifecycle: inquiry → quote → confirm → deposit → delivery → close
Resource Booking	Book equipment, studios, venues alongside worker time

8.2.2 CRM & Lead Management

Feature	Description
Lead Capture Forms	Embeddable forms on profile/portfolio pages; leads flow into CRM pipeline
Client Database	Contact info, booking history, spend, communication log, tags
Pipeline Management	Inquiry → Qualified → Quoted → Negotiating → Booked → Retained
Automated Follow-ups	Triggered sequences on stage changes, quote expiry, post-event
Lead Scoring	CRMEngine scores probability-to-book; Oba recommends outreach timing

8.2.3 Contracts & Invoicing

Feature	Description
Automated Contracts	Template contracts auto-populated from booking details; custom clauses supported
E-Signatures	Integrated e-signature workflow via ProviderSystem (DocuSign, HelloSign)
Payment Reminders	Configurable deposit and balance due reminders via kogi-bank invoice system
Recurring Billing	Subscription/retainer billing with automated recurring invoice generation

8.2.4 Logistics & Communication

Feature	Description
Tour / Project Itinerary	Multi-stop itinerary builder with timeline view; links to bookings and assignments
Resource Allocation	Assign equipment, vehicles, crew to dates/times; conflict detection
Logistics Tracking	Equipment rentals, transportation, crew schedules — live status tracking
In-App Communication	Dedicated logistics room per booking; broadcast to all crew; real-time updates
Finance & Reporting	Real-time expense tracking vs. budget; P&L per booking; tax report generation

8.3 Community Module

Feature Area	Items
Feed & Social	Feed · Timeline · Posts · Spaces · Chat · Messages · Rooms
Resource Sharing	Resource sharing · Resource access · Resource shared economics
Group Economics	Cooperative treasury · Revenue sharing · Group investment pools · Mutual aid funds
Shared Portfolios	Group/team/org/collective/cooperative/federation portfolios · Portfolio collaborations
Initiatives	Portable benefits · Grants · Microfinancing · Equity crowdfunding · Crowdresourcing
Operations	Booking & scheduling · CRM · Logistics

8.4 Bank Module

Module	Features
Wallet & Accounts	Ledgers · Journals · Balances · Payments · Taxes · Orders · Invoices
Portable Benefits	HSA · SEP-IRA · 401k · PEP · Portable savings · Income protection · Emergency savings
Grants & Microfinancing	Grant registry · Applications · Microloan origination · Community lending pools
Group Economics	Cooperative treasury (multi-sig) · Revenue distribution engine · Mutual aid funds
Equity Crowdfunding	Reg CF campaigns · Revenue share · SAFE notes · Cap table · Campaign escrow
Crowdresourcing	Resource pooling accounts · Contribution ledgers · Attribution settlement

SECTION 7 · **BANKING, FINANCE & CAPITAL SYSTEM (kogi-bank)**

9. kogi-bank — Banking, Finance & Capital System

kogi-bank is the complete financial infrastructure of the Kogi platform. It manages personal and organizational wallets, portable benefits accounts, grant systems, group economics, cooperative treasury, and equity crowdfunding. Every financial instrument is a Portfolio Item carrying full metadata, governance, lifecycle management, and audit trails.

9.1 Personal Account Types

Account Type	Description	Primary Use
HSA Account	Health Savings Account — portable benefit, pre-tax health expense fund; rolls over annually	Medical, dental, vision expenses
Portable Savings Account	Gig platform contribution tracking account; receives % contributions from gig platforms	Managed alongside benefit providers (e.g., Stride LLC)
Income Protection Fund	Reserves for income replacement during missed work, disability, or occupational accident	Short-term disability, sick pay, income gap coverage
Professional Development Account	Portable education and training fund; receives contributions from platforms or self	Certifications, courses, training programs
Emergency Savings Account	Rapid-access emergency buffer, separate from general savings	Sudden expenses, income interruptions
Retirement Account	SEP-IRA, 401k, PEP — portable retirement savings with tax advantages	Long-term retirement accumulation
Operating Wallet	Day-to-day operating expenses, payroll, and business costs	Operational expenses
Investment Wallet	Capital allocated for investment activities and portfolio building	Investments and capital deployment
Tax Reserve Wallet	Automated tax reserve computed from income events	Quarterly estimated tax payments

9.2 Portable Benefits System (KBNF-BEN)

The kogi-bank Portable Benefits System manages worker-centered benefits that remain with the individual. Every benefit account is a Portfolio Item carrying full metadata, governance, lifecycle management, and audit trails.

9.2.1 Platform Contribution Tracking Flow

Step	Action
1	Platform posts earnings event → kogi-bank PortableContributionEvent
2	BenefitsEngine computes contribution amount (e.g., 4% of pre-tip earnings)
3	Ledger entry created: debit Platform Contribution Receivable → credit Portable Savings Account
4	Contribution posted to benefit provider (e.g., Stride LLC) via ProviderSystem API
5	Worker notified via Oba: "DoorDash contributed \$12.40 to your portable savings this week."

9.3 Grants & Microfinancing

9.3.1 Grant System

Component	Description
Grant Registry	Catalog of available grants from government, foundation, platform, and community sources; filtered by eligibility, category, amount, and deadline
Grant Application	Structured workflow linked to a Portfolio Program or Project; supporting documents attached as Portfolio Artifacts
Grant Fund Disbursement	Awarded funds deposited to a dedicated Grant Account; disbursed against milestones per grant agreement
Grant Reporting	Impact and milestone reports auto-generated from Portfolio System data; submitted to grant providers on schedule
Community Grant Pools	Organizations and collectives create shared grant pools funded by member contributions; governed by governance proposals
GrantEngine Integration	kogi-engine GrantEngine scores grant matches by eligibility, portfolio strength, and historical award data

9.3.2 Microfinancing System

Component	Description
Microloan Marketplace	Workers and orgs post loan requests; community lenders fund fractional amounts (crowdlending)
Credit Scoring	kogi-engine RiskEngine computes platform credit score from portfolio activity, income, repayment history
Loan Terms Negotiation	Lender proposes terms; borrower accepts or counter-proposes via Exchange Deal mechanics
Repayment Tracking	Automated repayment schedule; kogi-bank processes payments and distributes pro-rated returns to lenders
Community Lending Pools	Collectives establish shared pools; members contribute capital and access loans at preferential rates
Collateral Support	Portfolio assets (IP, equity stakes, revenue streams) pledged as collateral; locked in kogi-bank escrow

9.4 Group Economics

Group Economics encompasses all collective financial structures and instruments on the Kogi platform — from cooperative treasury management and shared revenue distribution to group investment pools and collective resource acquisition.

Instrument	Description	Governance
Cooperative Treasury	Multi-sig controlled shared fund for operations, payroll, and investment	Multi-sig + governance vote for large disbursements
Revenue Sharing Pool	Collective revenue aggregated and distributed by formula (hours, equity, contribution)	Automated computation + manual approval
Group Investment Pool	Members collectively deploy capital into portfolio assets or equity campaigns	Governance proposal required
Mutual Aid Fund	Peer-support pool for member emergencies; funded by contributions	Claims reviewed by stewards
Resource Pooling Account	Shared account for collectively purchasing and managing shared resources	Majority approval above threshold
Crowdresourcing Pool	Community-sourced contributions of labor, assets, knowledge; attribution tracked	Contribution ledger with provenance
Collective Escrow	Multi-party escrow for group deals, shared purchases, or collaborative project funding	M-of-N multi-sig release

9.4.1 Revenue Distribution Engine — Cooperative Payroll Flow

Step	Action
1	Treasurer or Oba triggers distribution computation at period end
2	BenefitsEngine computes each member's share: hours contributed (60%) + equity share (30%) + governance bonus (10%)
3	Distribution summary posted as Governance Proposal for member approval vote
4	Approved → batch payout transaction set prepared
5	Multi-sig approvers confirm → batch executed atomically
6	All member Payroll Wallets credited simultaneously → Payroll Journal updated
7	Oba notifies each member: "Q1 distribution of \$X has been deposited."

9.5 Equity Crowdfunding Infrastructure

Component	Description
Campaign Types	Equity (Reg CF, Reg A+), Revenue Share, SAFE Note, Convertible Note, Donation, Reward-based
Cap Table Management	Investor equity positions tracked in kogi-bank Equity Accounts; vesting enforced; dilution computed on new rounds
Escrow Management	Campaign funds held in Campaign Account/Escrow Reserve; auto-disbursed on success; auto-refunded on failure

Component	Description
Revenue Share Engine	For revenue-share instruments: automated periodic computation and distribution of investor returns
Regulatory Compliance	Reg CF limits enforced; accredited investor verification hooks via ProviderSystem (Parallel Markets, Carta)
Investor Portal	Investor view of equity positions, expected returns, distribution history, portfolio performance

SECTION 8 · MARKETPLACE & EXCHANGE

10. Marketplace & Exchange

The Kogi Marketplace and Exchange are the economic layer of the platform — connecting independent workers with clients, investors, and opportunities, and enabling the trading of portfolio assets, labor, capital, and financial instruments.

10.1 Grants Marketplace

Feature	Description
Grant Registry	Searchable catalog of government, foundation, platform, and community grants; filtered by eligibility, category, amount, deadline, entity type
Grant Listings	Grant providers list grants with eligibility criteria, required documents, and review timeline
Grant Application Flow	Grant discovery → eligibility check → application → document submission → review → award; all linked to Portfolio Program/Project
Grant Fund Disbursement	Awarded funds routed through kogi-bank Campaign Account; milestone-based disbursement
Community Grant Pools	Organizations and collectives create shared grant pools; governed by community governance proposals
Grant Impact Reporting	Auto-generated impact reports from Portfolio System analytics; submitted on schedule to providers

10.2 Crowdfunding

Feature	Description
Crowdfunding Listings	Portfolio owners open items for community contribution; listed with contribution type, goal, and reward rules
Contribution Types	Labor (hours, tasks), Capital (monetary), Assets (designs, code, data), Knowledge (research, strategy, documentation)
Contribution Ledger	All contributions recorded with attribution in Portfolio EventLog; weight computed by CollaborationEngine
Reward Rules	Configurable: equity allocation, revenue share, direct payment, recognition badge, platform credits
Steward Review	Configurable gate: AutoAccept StewardReview GovernanceVote
Analytics	Contribution velocity, contributor diversity, goal progress, open gaps — surfaced in Portfolio Dashboard

10.3 Shared Portfolios & Portfolio Resource Exchange

Listing Type	Description
Shared Portfolio Showcase	A group portfolio published for community visibility; followers and investors can track progress
Collaborative Project Listing	A portfolio project open to new contributors; lists skills needed, contribution types, and reward model
Portfolio Resource Listing	Specific resources (templates, playbooks, tools, datasets) shared with or sold to the community
Federation Portfolio	Cross-organization portfolio open to aligned orgs for joint investment or resource contribution
Crowdfunded Portfolio Program	A portfolio program raising capital via equity, revenue share, or donation instruments

10.3.1 Tradeable Resource Types

Resource Type	Exchange Mechanism	Settlement
Portfolio Templates & Playbooks	Fixed-price listing or auction; buyer gains fork rights	kogi-bank instant settlement
Intellectual Property / Licenses	License agreement deal flow; DD report for IP assets	Escrow-backed settlement
Portfolio Equity Stakes	Secondary market offer/bid for equity in projects or orgs	Governance approval + cap table update
Revenue Streams	Forward-sale of future revenue share; exchange pricing via MatchEngine	kogi-bank revenue routing
Tool Integrations	Toolbox and toolchain configurations listed and traded	Fork + provider credential transfer
Data & Datasets	Research, analytics datasets, model outputs with licensing terms	Access-key settlement via kogi-bank

10.4 Booking, CRM & Logistics

10.4.1 Booking & Scheduling

Feature	Description
Bookable Profiles	Worker and org profiles set as 'bookable'; clients browse and initiate booking requests
Availability Calendar	Public calendar; client-facing booking widget for self-service scheduling
Booking Request Flow	Client inquiry → worker quotes → client accepts → deposit collected via kogi-bank → confirmed
Multi-Worker Booking	Book a team for multi-role events; conflict detection across all assigned workers
Resource Booking	Bundle equipment, venue, or other resources into a booking package

Feature	Description
Booking Analytics	Conversion rate, utilization, average deal size, peak demand periods

10.4.2 CRM & Lead Management

Feature	Description
Lead Capture	Marketplace profile inquiry forms feed directly into worker's CRM pipeline
Client Database	Full record: contact, booking history, spend, communication log, tags
Pipeline Stages	Inquiry → Qualified → Quoted → Negotiating → Booked → Retained → Lapsed
Automated Follow-ups	Configurable email/message sequences on stage transitions, quote expiry, post-booking
CRMEngine Integration	Scores lead probability-to-book; Oba recommends outreach timing and messaging
Retention Analytics	Repeat booking rate, lifetime value, churn risk; feeds Marketplace profile ranking

10.4.3 Logistics Tracking

Feature	Description
Tour / Event Itinerary	Multi-stop itinerary linked to Marketplace bookings; shareable with clients and crew
Equipment Tracking	Rental and owned equipment status per booking; return deadlines with automated reminders
Transportation Management	Vehicle assignments, routing, and mileage for multi-location bookings
Crew Schedules	Role-based crew assignments per booking; call times and wrap times distributed via logistics room
Live Status Updates	In-app logistics room per booking for real-time status broadcast to all stakeholders
Expense Tracking	Per-booking expense log reconciled against booking budget in kogi-bank

SECTION 9 · COMMUNITY, SPACES & SOCIAL SYSTEM

11. Community, Spaces & Social Management System

The Kogi Community, Spaces & Social Management System (KSPC · KRMS · KCOM · KORG) is the connective social tissue of the Kogi platform — connecting users to one another through shared spaces and rooms, social feeds and posts, events, organizations, teams, collectives, and cooperatives. Every post, space, or room is tied to portfolio components, and every action generates data that the Kogi Engine uses to personalize, match, and recommend.

11.1 Core Subsystems

Code	Subsystem	Primary Responsibility
KSPC	Spaces	Digital community spaces, organizations, teams, events
KRMS	Rooms	Real-time chat, DMs, group rooms, project rooms
KFED	Feed Service	Activity feeds, posts, timelines, notifications
KORG	Organizations	Autonomous orgs, collectives, cooperatives, federations
KSGR	Social Graph	Follow, subscribe, watch, connect relationships
KMTC	Match & Discovery	Worker matching, space recommendations, talent discovery
KENG-COM	Community Engine	Analytics, personalization, sentiment, engagement scoring

11.2 Space Types

Type	Description	Example
Community	Open or invite-only community around a topic or interest	Real Estate Investors Community
Team	Collaborative working group, squads, tribes, guilds, chapters	Design Squad Alpha
Organization	Formal legal or operational entity (LLC, Corp, Trust, Fund)	Pamoja Capital LLC
Collective	Informal resource-sharing group, skill pool	Freelance Dev Collective
Cooperative	Member-owned economic cooperative	Worker-Owned Creative Coop
DAO / Autonomous Org	Decentralized autonomous organization with on-chain governance	Protocol Builders DAO
Federation	A network of linked Spaces, Collectives, or Orgs	Pamoja Federation
Project Space	A Space tied to a specific portfolio project or program	Brand Launch 2026 Space

Type	Description	Example
Private Studio	Personal creative or research environment	Jordan's Innovation Lab

11.3 Space Data Model

Field	Description
space.id (UUID)	Platform-unique identifier
space.type	community team organization collective cooperative dao federation project studio
space.name	Display name of the Space
space.handle	Unique @handle for discovery and mention
space.visibility	public private protected invite-only
space.members[]	Member roster with roles: owner admin mod member contributor viewer guest
space.rooms[]	Linked Room component IDs
space.events[]	Linked Event component IDs
space.portfolio_links[]	Linked PortfolioComponent IDs (projects, programs, assets)
space.policies[]	Governance and access policy IDs
space.analytics	Engagement metrics, member growth, reach, sentiment scores

11.4 Events System

Event Type	Description
Meeting	Scheduled video or audio call; integrates with Zoom, Google Meet
Conference	Multi-session structured gathering; has a schedule, multiple event rooms
Workshop	Interactive skill-building session with materials and exercises
Mastermind	Peer-to-peer knowledge sharing group session
AMA (Ask Me Anything)	Open Q&A session hosted by a user or organization
Launch	Product, project, or portfolio launch event
Demo Day	Showcase of projects, products, or portfolio items
Governance Vote	Formal voting event for an organization; tied to a Proposal

11.5 Organization Types (KORG)

Type	Description
Autonomous Organization	Formally chartered organization with governance, treasury, and membership

Type	Description
Collective	Informal resource-sharing or skill-pooling group
Cooperative	Member-owned economic cooperative with democratic governance and revenue-sharing bylaws
DAO	Decentralized autonomous organization with on-chain or platform-native governance
Team	Working group within a project or program
Guild	Skill or trade-based professional association
Federation	Cross-organization meta-entity enabling joint governance and shared portfolio
Council / Assembly	Deliberative governance body for collectives or cooperatives

11.6 Content Visibility Model

Visibility Level	Description	Who Can See
public	Visible to all users and unauthenticated visitors	Anyone
followers	Visible to followers and subscribers	Followers, subscribers
protected	Visible to approved followers	Approved followers
space	Visible to Space members only	Space members
organization	Visible to org members only	Org members with appropriate role
private	Visible only to explicitly mentioned parties	Tagged users, owner

11.7 MVP Feature Scope

Feature	Module	MVP Priority
Space creation and management	KSPC	P0 — Core
Space member management (join, invite, roles)	KSPC	P0 — Core
Direct messaging (1:1 rooms)	KRMS	P0 — Core
Group rooms	KRMS	P0 — Core
Post creation and feed	KFED	P0 — Core
Follow / unfollow users and spaces	KSGR	P0 — Core
Like, comment, share on posts	KFED	P0 — Core
Notifications (mentions, replies, alerts)	KFED	P0 — Core
Basic organization creation (collective, team)	KORG	P1 — High

Feature	Module	MVP Priority
Activity feed per space	KFED	P1 — High
Event creation and RSVP	KSPC	P1 — High
Portfolio showcase posts	KFED	P1 — High
Space discovery / explore	KMTC	P1 — High
Governance proposals and voting	KORG	P2 — Medium
DAO / cooperative governance model	KORG	P2 — Medium
AI post drafting (Oba)	AI Layer	P2 — Medium
Collaboration matching	KMTC	P2 — Medium
Federation of spaces and orgs	KSPC/KORG	P3 — Later

SECTION 10 · DATA & INTELLIGENCE ENGINE (kogi-engine)

12. kogi-engine — Data & Intelligence Engine

kogi-engine is the central data-processing, analytics, and intelligence platform of the Kogi system. It consumes all platform event streams, runs multi-dimensional analytics, executes graph traversals on the portfolio dependency graph, powers personalized recommendations and content delivery, drives worker and resource matching, optimizes system resource allocation, interprets and optimizes SQL queries, and continuously assesses portfolio risk and health. Every sub-engine exposes a unified interface through the KogiEngine facade, accessible via gRPC, REST HTTP, and interactive CLI.

Property	Value
Package	kogi.engine
Language	Scala 3 (primary) · compiled on JVM
Interfaces	gRPC (EngineGrpcServer) · HTTP REST (GraphEngineAPIServer) · CLI (KogiEngineCli, GraphEngineCLI)
Status	MVP — all sub-engines implemented and integrated
gRPC Port	9100 (default)

12.1 Sub-engine Map

Sub-engine	File	Core Responsibility
KogiEngine	KogiEngine.scala	Unified facade — single access point for all sub-engines
AnalyticsEngine	AnalyticsEngine.scala	Event ingestion, module metrics, host metrics, portfolio signals, system snapshots, recommendations/discover/explore
TelemetryEngine	TelemetryEngine.scala	Platform data envelope processing, flow ledger, component flow stats, snapshot generation
DataStreamingEngine	DataStreamingEngine.scala	In-memory event stream, pub/sub subscriptions, profile/module/time-window queries
RecommendationEngine	RecommendationEngine.scala	User profiling, collaborative/content-based/hybrid/contextual/cold-start recommendations, feedback loop
PersonalizationEngine	PersonalizationEngine.scala	Content delivery adaptation, user segmentation, A/B experiments, multi-armed bandit, persona lifecycle
MatchEngine	MatchEngine.scala	Multi-dimensional matching of users, portfolio components, resources, analytics artifacts
GraphEngine	GraphEngine.scala	Dependency graph traversal, cycle detection, topological sort, critical path (CPM), graph diff

Sub-engine	File	Core Responsibility
RiskEngine	RiskEngine.scala	Portfolio health scoring, persona-aware risk adjustment, risk optimization plans
OptimizationEngine	OptimizationEngine.scala	System resource optimization recommendations, persona-aware workload tuning
SearchEngine	SearchEngine.scala	Full-text + tag/metadata indexing, personalized re-ranking
QueryEngine	QueryEngine.scala	SQL normalization, cost estimation, query optimization, persona-aware row caps
AllocationEngine	AllocationEngine.scala	Bid scoring and allocation selection for game listings
IncentiveEngine	IncentiveEngine.scala	KP ledger, incentive profiles, streaks, reward application
GameEngine	GameEngine.scala	Listings, bids, allocation orchestration, incentive application
BenefitsEngine	(new)	Benefits eligibility scoring, contribution tracking, coverage optimization, tax optimization
GrantEngine	(new)	Grant matching, eligibility scoring, impact tracking, community grant pool management
CrowdfundingEngine	(new)	Campaign scoring, investor matching, equity modeling, Reg CF compliance checks
CollaborationEngine	(new)	Shared portfolio coordination, contribution attribution, conflict resolution, health scoring
BookingEngine	(new)	Scheduling optimization, conflict detection, resource allocation, utilization analytics
CRMEngine	(new)	Lead scoring, pipeline management, follow-up automation, retention analytics
LogisticsEngine	(new)	Itinerary planning, resource routing, crew scheduling, expense tracking

12.2 Data Flow Architecture

Platform Events → TelemetryEngine.ingestEnvelope(PlatformDataEnvelope) → normalizes source/target/hops → writes DataFlowLedgerEntry → splits into:
 AnalyticsEngine.ingest(StreamEvent) [portfolio signals], AnalyticsEngine.ingestModuleMetric() [latency/CPU/queue], AnalyticsEngine.ingestHostMetric() [host saturation],
 RecommendationEngine.recordInteraction() [feedback loop] →
 AnalyticsEngine.systemSnapshot() → calls RiskEngine.score(signal) + RecommendationEngine (discover/explore cards) → TelemetryEngine.snapshot() → EngineFlowSnapshot → returned via gRPC / REST / CLI to consuming services.

12.3 Interface Layer

Interface	File	Protocol	Key Commands / Endpoints
gRPC Server	EngineGrpcServer.scala	gRPC / protobuf	Control · Status · Ingest · Snapshot
HTTP REST Server	GraphEngineAPIServer.scala	HTTP/1.1 JSON	GET /health · POST /load · GET /nodes · GET /edges · GET /impact · GET /topo · GET /critical · GET /cycles · GET /diff
Engine CLI	KogiEngineCli.scala	stdin/stdout (JSON)	control, status, ingest, snapshot, query, search, match-profiles-resources, graph-impact, graph-closure, graph-cycles, graph-critical, graph-diff
Graph CLI	GraphEngineCLI.scala	stdin/stdout (text+JSON+DOT)	impact · closure · rdeps · topo · critical · cycles · diff · nodes · edges · export

12.4 Key Data Types

Type	Package Role	Key Fields
StreamEvent	AnalyticsEngine input	profileId · module · eventType · productivity/cashFlow/collaboration/riskDelta
PortfolioSignal	Risk/Analytics shared	productivity · cashFlow · collaboration · risk (all 0–100)
UserInteraction	RecommendationEngine	userId · itemId · interactionType · value · sessionId · context
UserProfile	RecommendationEngine	userId · interactionHistory · preferenceVector · dominantCategories · persona
GraphEdge	GraphEngine	from · to · edgeType(Dependency Hierarchy Relationship) · weight
MatchSubject	MatchEngine	sealed: UserSubject ComponentSubject ResourceSubject AnalyticsArtifact
PlatformDataEnvelope	TelemetryEngine input	id · flowId · source · target · topic · payload · timestampMs · hops

12.5 Sub-engine Configuration Defaults

Parameter	Default	Sub-engine
DataStreamingEngine.maxEvents	10,000	DataStreamingEngine
AnalyticsEngine.defaultWindow	250 events	AnalyticsEngine
AnalyticsEngine.defaultRealtimeWindowMs	5 minutes	AnalyticsEngine

Parameter	Default	Sub-engine
AnalyticsEngine.recommendationLimit	8	AnalyticsEngine
TelemetryEngine.maxEnvelopes	50,000	TelemetryEngine
SearchEngine.maxDocuments	50,000	SearchEngine
QueryEngine default row cap	1,000 rows	QueryEngine
QueryEngine PowerUser cap	10,000 rows	QueryEngine
EngineGrpcServer port	9100	EngineGrpcServer

12.6 Scaling Considerations

- In-memory state: all sub-engines currently use in-memory storage. Production deployment should back StreamEvents and envelopes with Kafka, user profiles with Redis, and document indices with Elasticsearch.
- gRPC concurrency: the Netty-based gRPC server handles concurrent requests on its thread pool. High-throughput deployments should use actor-based concurrency (Akka).
- GraphEngine rebuild cost: every graph mutation rebuilds the full adjacency maps from the edge/node Seq. For graphs >10K nodes, use a mutable adjacency map with incremental updates.
- Multi-host federation: cross-host synchronization of recommendation profiles and graph state should use the Kafka event bus and CRDT-based merge strategies for distributed convergence.

SECTION 11 · GAME SYSTEM & MECHANISM ENGINE (kogi-game)

13. kogi-game — Game System & Mechanism Engine

Design Principle

Mechanism design is the engineering of economic games. Where economics asks 'what outcomes do given rules produce?', mechanism design asks 'what rules produce the outcomes we want?' kogi-game answers that question for every interaction on the platform — from a single gig bid to a multi-million dollar equity round.

kogi-game is the mechanism design and game engine of the Kogi Platform — the mathematical and computational substrate that governs how resources, capital, labor, portfolio components, and financial instruments are discovered, matched, priced, allocated, exchanged, and incentivized across the platform.

13.1 Four-Layer Architecture

Layer	Name	Responsibility	Implementation
L4 (Top)	Incentive Layer	Reputation, rewards, stakes, and long-run behavior shaping	IncentiveEngine — Go service + kogi-engine signals
L3	Allocation Layer	Deciding who gets what, when, at what price, in what order	AllocEngine — Rust core + Go service
L2	Matching Layer	Finding optimal pairings across all entity types	MatchEngine — Scala (kogi-engine) + Go service
L1 (Base)	Signal Layer	Collecting, normalizing, and distributing the data that all higher layers depend on	TelemetryEngine + AnalyticsEngine (kogi-engine)

13.2 Mechanism Design Principles

Principle	Application in kogi-game
Incentive Compatibility	Mechanisms are designed so the dominant strategy for every participant is to reveal their true preferences — honest bidding, accurate skill declaration, genuine availability — because truthful behavior maximizes their payoff.
Individual Rationality	No participant is made worse off by participating. Every mechanism guarantees the outside option is always at least as good as the worst outcome from participating.
Pareto Efficiency	Allocations are designed to exhaust all mutually beneficial trades before closing a market. No resource sits idle when a willing buyer and seller could be matched.

Principle	Application in kogi-game
Budget Balance	Platform fees and mechanism payouts are designed so the platform's net revenue from running a mechanism is non-negative without extracting rents that would deter participation.
Fairness & Non-Discrimination	Matching algorithms do not discriminate on protected characteristics. Scoring functions are transparent, auditable, and explainable to all participants.
Liquidity Design	Mechanisms account for thin markets (few participants) by using reservation prices, automated market makers, and fallback mechanisms to prevent market failure.
Dynamic Consistency	Mechanisms produce consistent outcomes over time — a rule that produces a good outcome today should not produce a bad outcome tomorrow due to strategic adaptation.
Transparency	Mechanism rules, scoring weights, and outcome explanations are available to participants. Opaque 'black box' allocations are avoided.

13.3 Game Types by Platform Context

Game Type	Platform Context	Mechanism
Assignment Game	Worker-to-gig matching, resource-to-project matching	Deferred Acceptance (Gale-Shapley) with multi-dimensional scoring
Double Auction	Exchange market — simultaneous buyers and sellers posting prices	Continuous Double Auction (CDA) with price-time priority order book
Vickrey Auction (VCG)	Single-item high-value asset sales, exclusive contracts	Second-price sealed-bid; incentive-compatible, truthful revelation
Combinatorial Auction	Bundled resource or asset packages, multi-resource project staffing	Iterative Combinatorial Auction (ICA) with proxy bidding
Cooperative Game	Cooperative revenue distribution, collective investment allocation	Shapley Value (Monte Carlo approximation for large cooperatives)
RFP / Reverse Auction	Client posts requirements; workers compete on price/quality	Multi-attribute scoring with buyer-defined rubric weights
Campaign Coordination	Equity crowdfunding, group-buy, resource pooling	Sequential commitment with milestone-gated disbursement

13.4 Entity Classes in the Game

Entity Class	Platform Types	Game Role
Labor	Independent workers, contractors, freelancers, gig workers, team members	Sellers of time, skills, and deliverables; bidders in RFP auctions
Capital	Investors, donors, lenders, campaign backers, grant providers	Allocators of financial resources; participants in investment matching

Entity Class	Platform Types	Game Role
Resources	Compute, licenses, materials, equipment, tools, data, templates	Supply-side of resource allocation games; inputs to production
Portfolio Components	Programs, Projects, Assets, Artifacts, Resources	Objects of investment, trade, and collaboration; generate signals
Financial Instruments	Equity, revenue-share notes, bonds, tokens, derivatives	Traded in kogi-exchange financial instruments market
Demand-side Entities	Buyers, hirers, organizations, cooperatives, teams	Create demand signals; fund escrow; initiate matching
Transactional Entities	Bids, Offers, Deals, Requests, Proposals, Contracts, Gigs, Tasks	Game moves — the atomic actions through which players interact
Campaigns	Fundraise, equity, group-buy, resource, bond, pre-sale	Coordination games — multiple buyers coordinating on shared purchase or investment
Collectives/Cooperatives	Worker collectives, DAOs, federated entities	Cooperative games — designing fair distribution rules within groups

13.5 End-to-End Game Flows

13.5.1 Worker Wins a Contract via Reverse Auction RFP

- Buyer posts RFP in kogi-marketplace with scope, budget, and scoring rubric (30% price / 40% skills / 20% reputation / 10% timeline)
- MatchEngine broadcasts to top-50 worker matches; Oba notifies each in the next morning briefing
- Worker reviews RFP in their Proposal Center; Oba drafts a competitive proposal with market-rate pricing guidance
- Worker submits proposal; AllocEngine scores it against the buyer's rubric and ranks it against all proposals
- Buyer reviews ranked shortlist; Oba summarizes top-3 proposals with key trade-offs
- Buyer accepts top-ranked proposal; Deal Room opens; milestone schedule negotiated
- Deal agreed; kogi-bank provisions escrow; project appears in worker's kogi-portfolio
- On deal close: worker earns KP + Delivery Reliability update; buyer earns KP + Financial Integrity confirmation

13.5.2 Cooperative Closes a Sprint and Distributes Patronage

- Cooperative's sprint closes; all task completions recorded in kogi-portfolio
- AnalyticsEngine computes per-member contribution signals (hours, quality scores, deliverables)
- AllocEngine runs Monte Carlo Shapley approximation on contribution vectors
- Oba presents distribution plan to cooperative governance
- Governance vote initiated; members vote via kogi-organizations; quorum reached
- Vote passes; kogi-bank executes batch payroll to all member Payroll Wallets atomically
- All members earn Governance Participation KP; distribution logged in cooperative's ledger

SECTION 12 · AI AGENT & ASSISTANT WORKFLOWS (OBA)

14. AI Agent & Assistant Workflows

The Kogi platform is built around a single organizing idea: that AI agents and assistants should remove friction from every part of a user's professional life — from managing projects and finances, to hiring talent, closing deals, and governing cooperative organizations. Oba is Kogi's AI assistant, running on the Sambara agent system and ImaniOS, powered by Echuya-LLM.

14.1 Core AI Design Principles

Principle	Description
Cross-module awareness	The AI holds a unified context window spanning all modules simultaneously — project status, wallet balance, open invoices, OKR progress, and governance votes are all visible at once.
Proactive, not reactive	Agents run on a continuous monitoring loop. They surface time-sensitive issues and present suggested actions the moment something requires attention — not when the user happens to open the relevant screen.
Tool-native execution	When a user approves an action, the agent executes it natively within Kogi: sending inquiries, creating invoices, drafting governance proposals, updating sprint boards, or triggering escrow releases.
Approval-first by default	Every consequential action requires explicit user confirmation. Agents suggest and prepare; users decide.
Full audit trail	All agent actions are logged with timestamps, tool names, inputs, and outcomes. Users can inspect or replay any agent decision from the History panel at any time.
Graceful degradation	If an agent action fails, it clearly reports the failure and proposes an alternative path, rather than silently stopping.

14.2 AI Workflow Summary

Module / Screen	Primary AI Workflow	Tools Invoked	User Value
AI Agent	Morning briefing & cross-module insight	read_portfolio, scan_invoices, search_marketplace	One dashboard replaces 10 manual module checks every morning
Work Board	Sprint unblocking via Marketplace	monitor_board, search_marketplace, draft_inquiry	Blocked stories resolved in hours rather than days
Exchange	Invoice recovery & overdue management	scan_invoices, draft_email, calculate_cashflow	Overdue payment recovery is proactive and documented

Module / Screen	Primary AI Workflow	Tools Invoked	User Value
Governance	Proposal drafting, vote monitoring & execution	draft_proposal, notify_members, execute_proposal	Full governance cycle with audit trail and zero missed votes
Investor Outreach	Pipeline management & personalized outreach	draft_email, update_pipeline, prepare_materials	Fundraising velocity increases; no lead goes cold
Community	Content amplification & collaboration matching	draft_post, analyse_engagement, match_collaborators	Showcase visibility and collaboration opportunities grow continuously
Studio (Idea→Asset)	Stage-gate AI at every transition	draft_spec, write_tests, publish_asset	Ideas reach market faster with systematically higher quality
Legal	Contract & compliance monitoring	monitor_contracts, draft_renewal, track_compliance	Zero missed deadlines; renewals prepared 14 days early
OKR & Analytics	Velocity tracking & intervention planning	calculate_velocity, suggest_interventions, digest	Strategy remains connected to daily execution at all times
Capital Exchange	Round tracking, cap table & distributions	track_round, model_captable, execute_distribution	Raise coordination and investor communication automated
Bank & Treasury	Reserve monitoring & tax automation	monitor_wallets, calculate_tax, forecast_cashflow	CFO-level treasury intelligence without dedicated finance staff

14.3 Morning Portfolio Briefing Workflow

Every morning when a user opens Kogi, the AI Agent automatically runs a portfolio-wide review and surfaces the most important items requiring attention. This replaces the need to manually check every module.

#	Phase	What the Agent Does	Tools Invoked	Outcome
1	Portfolio scan	Reads all active projects, sprint statuses, story blockers, and milestone dates across the Work Board	read_portfolio, read_wbs	Full project snapshot
2	Financial review	Checks wallet balance, scans open invoices for overdue items, reviews escrow holds, flags pending payouts	read_exchange, read_invoices	Financial health score
3	OKR delta check	Compares current revenue and delivery metrics against quarterly	read_okrs, calculate_velocity	Risk flags raised

#	Phase	What the Agent Does	Tools Invoked	Outcome
		OKR targets, calculates velocity and projected shortfall		
4	Governance scan	Checks for active votes closing within 48h where user has not yet voted, summarizes each proposal	read_governance, summarise_proposals	Vote alerts queued
5	Marketplace match	Searches for contractors or resources that could resolve any blockers detected in step 1	search_marketplace, rank_candidates	Candidates found
6	Briefing composed	Ranks all findings by urgency, composes a prioritized briefing with 3–5 actionable insights and a suggested action for each	create_brief, rank_actions	Briefing delivered
7	User approves	User reviews suggested actions and approves or modifies. Agent executes approved actions immediately	execute_actions, log_audit	Actions executed

SECTION 13 · PLATFORM TOOLS

15. Platform Tools

Kogi Platform Tools are purpose-built, AI-assisted utilities that help independent workers, creatives, professionals, and organizations generate, design, manage, and publish the documents, content, and assets they need to operate and grow. All tool outputs are automatically saved as versioned Portfolio Artifacts, linked to relevant work items, and reusable across every module.

15.1 Tools Catalog

Tool ID	Tool Name	Category	Key Integration
T-01	Resume & Work-Portfolio Highlights Builder	Identity & Presentation	Portfolio System · AnalyticsEngine · MatchEngine
T-02	Bio & Elevator Pitch Generator	Identity & Presentation	Profile · PersonalizationEngine
T-03	Press Kit Builder	Identity & Presentation	Portfolio Artifacts · CRM
T-04	Brand Kit & Social Media Kit Builder	Identity & Presentation	Profile · Content Calendar · Publishing
T-05	Proposal & Pitch Deck Generator	Business Development	CRM · Portfolio · kogi-bank · Contract (T-09)
T-06	Rate & Pricing Calculator	Business Development	MatchEngine · kogi-bank Income Data
T-07	Scope of Work (SOW) Builder	Business Development	Contracts (T-09) · Work Management Backlog
T-08	Client Onboarding Toolkit	Business Development	CRM · Contracts · Invoicing · Portfolio Space
T-09	Contract Template Generator	Financial & Legal	kogi-bank · e-Signature · Portfolio EventLog
T-10	Invoice & Payment Builder	Financial & Legal	kogi-bank Ledgers · CRM · Contract (T-09)
T-11	Tax & Income Estimator	Financial & Legal	kogi-bank · BenefitsEngine · Portable Benefits
T-12	Grant Application Builder	Financial & Legal	GrantEngine · Portfolio · kogi-bank Grants
T-13	Project Brief & Charter Generator	Work Planning	Work Management WBS · Portfolio · RiskEngine
T-14	OKR & Goal-Setting Tool	Work Planning	Work Management Analytics · Portfolio Goals

Tool ID	Tool Name	Category	Key Integration
T-15	Retrospective & Review Facilitator	Work Planning	Work Management Backlog · Portfolio Artifacts
T-16	Cooperative Charter Builder	Community & Orgs	Org System · kogi-bank Treasury · Governance
T-17	Initiative & Campaign Planner	Community & Orgs	Community Initiatives · Work Management · Bank
T-18	Equity Crowdfunding Campaign Builder	Community & Orgs	CrowdfundingEngine · kogi-bank Escrow · Cap Table
T-19	Skills Mapper & Gap Analyzer	AI Tools (Oba)	MatchEngine · Professional Dev Account · Profile
T-20	Career Path & Rate Benchmarking Advisor	AI Tools (Oba)	MatchEngine · AnalyticsEngine · OptimizationEngine
T-21	Content Repurposer & Publishing Kit	AI Tools (Oba)	Portfolio Artifacts · AnalyticsEngine · Brand Kit
T-22	Opportunity Matcher & Pitch Personalizer	AI Tools (Oba)	MatchEngine · CRM · Proposals (T-05)

15.2 Shared Tool Architecture

Layer	Description
Input Layer	Form-driven or conversational (Oba-mediated) data collection; pre-populated from Portfolio, Profile, and kogi-bank data where available
Generation Engine	AI generation via Oba (Claude-powered); template-based composition; rule-based data transformation
Output Layer	Produces: PDF export · DOCX export · Portfolio Artifact · Shareable link · Direct publish to Profile / Marketplace / Community
Storage Layer	All outputs saved as versioned Portfolio Artifacts; linked to relevant Portfolio Components; full history retained
Analytics	View counts, share rates, conversion tracking (e.g., proposal-to-close), download events tracked per artifact
Personalization	PersonalizationEngine tunes tone, format, and content emphasis based on user Persona, work history, and target audience context

15.3 Template & Asset Library

Library Component	Description
Document Templates	30+ resume templates · 20+ proposal layouts · 15+ contract types · 10+ pitch deck formats · 8+ press kit layouts
Clause Library	300+ contract clauses by type and jurisdiction; community-tagged; Oba-curated for clarity and completeness

Library Component	Description
Achievement Phrases	STAR-format achievement phrase bank organized by skill domain and impact type; used by resume and bio tools
Brand Starters	10+ brand voice profiles by persona type; color palette starters; typography pairings
Prompt Templates	Curated Oba prompt templates for each tool; community-improvable; A/B tested for output quality
Community Contributions	Members share improved templates back to the library; steward-reviewed; attribution credited to contributor

SECTION 14 · CROSS-SYSTEM INTEGRATIONS & ENGINE MAP

16. Cross-System Platform Integrations

KOGI-RMS does not operate in isolation. Resource data flows bidirectionally across every Kogi subsystem. The following integration matrix defines the primary data exchange surfaces between all platform modules.

Kogi System	Integration Surface	Resource Flow
Portfolio System	All KOGI-RMS resources are Portfolio Components; full CRDT, EventLog, governance, and analytics framework inherited	Bidirectional: all resource mutations sync to Portfolio EventLog
kogi-bank	Financial resources (accounts, instruments, grants, invoices) originate in kogi-bank; budgets and expenses flow back to resource dashboards	Bidirectional: resource financial actions write to bank ledgers
Marketplace	Commerce resources (gigs, offers, deals, proposals, bids) published to Marketplace; applications flow into resource assignment workflows	Outbound: publish; Inbound: inquiries, bids, applications
Exchange	Portfolio assets, equity stakes, revenue streams, IP, data — listed and traded on Exchange; settlement back to kogi-bank	Outbound: list; Inbound: bids, offers, settlement events
Community & Spaces	Resources shared to community spaces; crowdresourced contributions; initiative enrollment; organization membership and governance	Bidirectional: publish to community; contributions flow back
Work Management (WMS)	Work execution resources (boards, stories, tasks, sprints) are dual-registered in RMS and WMS; analytics unified	Bidirectional: WMS and RMS share the same underlying data model
Developer / API	All resource types accessible via REST and gRPC; webhook notifications on lifecycle transitions; SDK helpers for typed resource creation	Full CRUD via kogi-api; event subscriptions
kogi-community	All community events streamed to engine; feed ranking, match scoring, recommendation, analytics all powered by engine	Community → Engine (stream) · Engine → Community (score)

16.1 Complete Engine-to-Module Matrix

Engine	Role in Resource Management
AnalyticsEngine	Computes all engagement, performance, and usage analytics for every resource type
TelemetryEngine	Real-time event streaming: resource state changes, user activity, system events

Engine	Role in Resource Management
RecommendationEngine	Personalized resource discovery; 'resources you may need'; cross-type recommendation
PersonalizationEngine	Persona-aware UI surfaces; priority sorting of resources per user profile
MatchEngine	Worker-to-gig, resource-to-need, investor-to-campaign matching across resource types
GraphEngine	Dependency graph computation; resource relationship traversal; impact analysis
RiskEngine	Risk scoring for commitments, dependencies, financial instruments, and org governance
OptimizationEngine	Resource allocation optimization; capacity planning; portfolio health scoring
SearchEngine	Full-text and structured search across all resource types, metadata, and content
AllocationEngine	Human and physical resource allocation; conflict detection; over-allocation warnings
IncentiveEngine	Contribution attribution; reward computation; cooperative revenue distribution
CollaborationEngine	Shared portfolio coordination; concurrent edit conflict resolution; attribution weighting
BenefitsEngine	Benefits eligibility scoring for worker types; contribution tracking; coverage optimization
GrantEngine	Grant matching by eligibility, portfolio strength, and historical data
CRMEngine	Lead scoring and pipeline management for commerce resources
BookingEngine	Scheduling optimization; conflict detection; resource calendar coordination
LogisticsEngine	Physical resource routing; equipment tracking; crew schedule optimization
DataStreamingEngine	Real-time data export; BI connector feeds; event bus for third-party integrations
GameEngine	Gamification: contribution points, badges, streaks, community leaderboards

SECTION 15 · BRAND & DESIGN SYSTEM

17. Brand & Design System

17.1 Color Palette

Token	Hex	Usage
Core Blue	#1D6FEA	Trust, intelligence — primary action color
Ink Black	#0A0D14	Clarity, depth — primary text
Soft Gray	#F2F4F7	Calm background — primary background
Steel Gray	#A6AEB8	Neutral separators — borders and muted elements
Electric Teal	#19C6C6	AI highlights — Oba and intelligence surfaces
Warm Yellow	#F8C841	Attention, alerts — warnings and notifications
Emerald Green	#21B77E	Success, validation — completion and positive states
Coral Red	#FF5A5F	Critical alerts — errors and critical warnings

17.2 Typography

Role	Font
Primary	Inter or Roboto
Secondary	Source Sans
Monospace (Dev tools)	JetBrains Mono

17.3 Design Principles

- Clarity Over Complexity — Every screen should feel simple, even when the backend is complex
- Flow-Based UX — Every feature guides users from intent → action → result
- Modular Blocks — Reusable UI patterns reflecting the modular architecture
- Calm, Focused, Minimal — Reduce clutter; foreground the user's goals
- Intelligence as Companion — AI suggestions are offered, not demanded

17.4 Brand Voice

Tone: Smart · Clear · Supportive · Insightful — always calm, structured, and confident. Never: Loud, chaotic, overly casual, or jargon-heavy.

Example Microcopy

- "You're doing great. Let's optimize your next sprint."

- "Here's a clearer financial picture for this month."
 - "Would you like me to prepare a proposal draft for this client?"
 - "Here's what changed today."
-

SECTION 16 · PRODUCT ROADMAP

18. Product Roadmap

18.1 Phase 1 — MVP (Months 1–4): Core Foundation

Component	Features
KOGI-Host	Auth, Identity, RBAC, Settings
Shared Services	Feeds, Notifications, Search, Files
kogi-portfolio	Portfolio Items, Workspaces, Files, Dashboards
kogi-project	Tasks, Sprints, Boards, Backlogs
kogi-chat	Messaging, Alerts
AI Engine (Basic)	Recommendations, Feed Ranking
Automation (Basic)	Trigger → Action Workflows

Goal: Functional portfolio + project manager with messaging — the minimal independent worker OS.

18.2 Phase 2 — Worker Operations (Months 4–6)

Component	Features
kogi-gig	Scheduling, Finances, Documents, Benefits, Contact Books
kogi-pay / kogi-bank	Billing, Invoices, Digital Wallet, Payment Tracking, Portable Benefits
kogi-design / kogi-studio	Idea → Prototype Creation
AI Engine	Matching Engine, Insights, Predictions
Automation	Multi-step Cross-app Workflows

Goal: Self-contained independent worker operating system.

18.3 Phase 3 — Network & Marketplace (Months 6–9)

Component	Features
kogi-community	Groups, Events, Posting, Followers, Activity Feeds, Spaces, Rooms
kogi-marketplace	Deals, Proposals, Reviews, Worker Matching, Booking & CRM
kogi-bank (expanded)	Grants, Microfinancing, Group Economics, Equity Crowdfunding, Portable Benefits
AI Engine (Advanced)	Prediction, Risk Analysis, Optimization, Semantic Search
kogi-dev	Extensions, SDK, Plugin Marketplace
KOGI-APPSTORE	App Discovery, Publishing, Distribution

Goal: Full ecosystem supporting global worker networks.

18.4 Phase 4 — Scaling & Intelligence (Month 12+)

- AI autonomous agents for portfolio, finances, and scheduling
 - Automated risk management and resilience
 - Global marketplace liquidity pools
 - Enterprise and organization extensions
 - Multi-portfolio orchestration
 - Advanced CMS/KMS integration
 - Full data observatory and BI suite
 - On-chain mechanism execution for high-value auctions and cooperative distributions
 - Cross-platform reputation portability via verifiable credential standard
 - Federated matching — MatchEngine operates across ShangoOS federates
 - Simulation environment (oru integration) for counterfactual strategy development
 - AI-autonomous market making — Oba agents post algorithmic bids to thin markets
 - Prediction markets for project outcomes and campaign forecasts
-

SECTION 17 · GLOSSARY & TERMINOLOGY REFERENCE

19. Glossary

Term	Definition
AllocationEngine	Kogi engine subsystem managing resource allocation, capacity planning, and conflict detection
AnalyticsEngine	kogi-engine subsystem computing portfolio signals, module metrics, host metrics, and system snapshots
Artifact	A typed Portfolio Item representing a document, file, design output, or generated asset; carries versioning, provenance, and access control
Backlog	An ordered list of work items awaiting refinement, prioritization, or sprint commitment
BenefitsEngine	Engine managing portable benefits eligibility scoring, contribution tracking, and coverage optimization
Booking Engine	Engine managing scheduling optimization, conflict detection, and resource calendar coordination
CollaborationEngine	Engine coordinating shared portfolio editing, contribution attribution, and concurrent conflict resolution
Cooperative	A worker-owned organization with democratic governance and revenue-sharing bylaws; managed via cooperative treasury in kogi-bank
CRDT	Conflict-free Replicated Data Type — the distributed data structure enabling safe concurrent edits across all resource types
CRMEngine	Engine managing lead scoring, pipeline management, follow-up automation, and retention analytics
Crowdresourcing	Community-sourced contributions of labor, assets, knowledge, and capital to a shared resource or project
DataStreamingEngine	kogi-engine subsystem managing in-memory event streams, pub/sub subscriptions, and time-window queries
Echuya-LLM	The language model powering Oba, the Kogi AI assistant
Epic	A large body of work decomposed into stories, fitting within a quarter or Program Increment
EventLog	An append-only, tamper-evident audit history attached to every Portfolio Component recording all mutations with actor, timestamp, and delta
Federation	A cross-organization meta-entity enabling joint governance, shared portfolio, and inter-org resource exchange between aligned organizations; also refers to the network of kogi-host nodes
GameEngine	Kogi engine subsystem managing gamification mechanics: contribution points, badges, streaks, and community leaderboards

Term	Definition
GraphEngine	kogi-engine subsystem for dependency graph traversal, cycle detection, topological sort, and critical path computation
GrantEngine	Engine matching grants by eligibility, portfolio strength, and historical award data
ImaniOS	The operating system on which Oba runs
IncentiveEngine	kogi-engine subsystem managing the KogiPoints ledger, incentive profiles, streaks, and reward application
jiwe seed	A single boot configuration artifact that can configure the full ShangoOS platform stack
KogiPoints (KP)	Platform-internal non-transferable credits earned through contributions, deliveries, governance participation, and referrals
LogisticsEngine	Engine managing itinerary planning, resource routing, crew scheduling, and expense tracking
MatchEngine	kogi-engine subsystem responsible for matching workers to gigs, resources to needs, and investors to campaigns
Multi-Sig Treasury	An organization bank account requiring M-of-N approver signatures for disbursements above a configured threshold
Oba	The Kogi AI assistant; runs on Sambara agent system and ImaniOS; powered by Echuya-LLM; surfaces recommendations, analytics, governance alerts, and optimization suggestions across all modules
OptimizationEngine	kogi-engine subsystem generating system resource optimization recommendations and persona-aware workload tuning
PersonalizationEngine	kogi-engine subsystem building user profiles, ranking content for personalized feed delivery, and managing A/B experiments
Portfolio Component	The base data model for all items on the Kogi platform; every resource is a Portfolio Component with CRDT versioning and EventLog auditing
Portable Benefits	Worker-centered benefits (health, retirement, PTO, insurance, professional development) that travel with the individual, not the employer
Program Increment (PI)	A SAFe-style planning interval comprising 5 sprints; the primary planning cadence for large programs
RecommendationEngine	kogi-engine subsystem providing user profiling, collaborative/content-based/hybrid recommendations, and feedback loop management
RiskEngine	kogi-engine subsystem scoring portfolio health, computing persona-aware risk adjustments, and generating risk optimization plans
Sambara	The agent coordinator system that runs Oba and orchestrates agents across qala, ume, kogi, and other ShangoOS components
SearchEngine	kogi-engine subsystem providing full-text + tag/metadata indexing with personalized re-ranking
ShangoOS	The combined operating system formed by kogi + qala + ume + oru + sambara + imewe + nandi + osyse

Term	Definition
Space (KSPC)	A named digital community environment; top-level social container on the Kogi platform
Story	The primary deliverable work unit: estimable, testable, and completable within a sprint
TelemetryEngine	kogi-engine subsystem processing platform data envelopes, maintaining flow ledger, and generating component flow stats
Timebox	A fixed time period bounding a set of committed work items (sprint, PI, quarter, custom duration)
VectorClock	A distributed data structure enabling CRDT-based conflict-free replication across platform nodes
WBS	Work Breakdown Structure — hierarchical decomposition of work from Theme to Task
ZOPA	Zone of Possible Agreement — the price range within which a deal between two parties is mutually beneficial; computed by AllocEngine from market rate data

KOGI

Independent Worker Operating System

Unified Platform Design Document · v3.0 · March 2026 · Confidential